LAPORAN PROYEK PEMROGRAMAN JARINGAN

CONNECT FOUR

PEMROGRAMAN JARINGAN C

Kelompok 1 :

Medericus Mundi Miseridityo	5025231169
Muhammad Baihaqi Dawanis	5025231177
Sadam Ali Rafsanjani	5025231025
Daffa Diandra Rizky A.	5025221261
Mochammad Irham Maulana	5025221251

DEPARTEMEN TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS

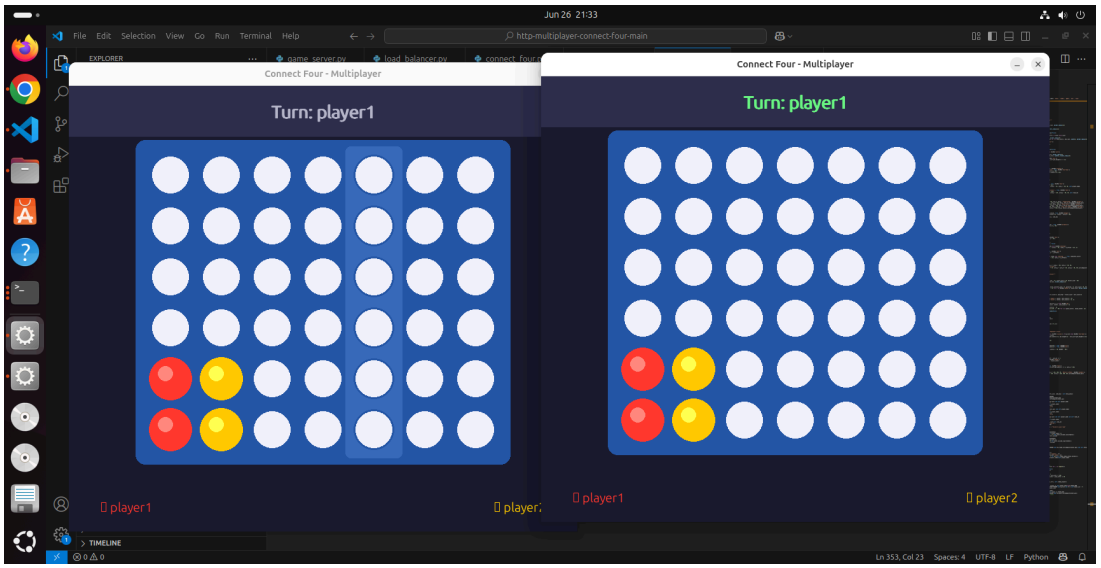
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2025

Daftar Isi

Daftar Isi....2
Deskripsi Game....3
1. Latar Belakang....3
2. Tujuan Proyek....3
3. Fitur Utama....3
4. Alur Permainan....3
5. Komunikasi Client-Server....5
6. Keunggulan Arsitektur....5
7. Teknologi yang Digunakan....5
8. Skenario Penggunaan....5
Informasi Final Project....7
1. Github Repository....7
2. Youtube....7
3. Pembagian Tugas....7
Protokol Multiplayer Game....8
1. Protokol....8
2. Mekanisme Kerja Client-Server....9
3. Diagram Alur Komunikasi....11
Arsitektur Implementasi....12
1. IP Address Client dan Server....12
2. Port TCP/IP yang Digunakan....12
3. Instruksi Menjalankan Server dan Client....12

Deskripsi Game



Gambar Game Connect Four

1. **Latar Belakang**

   Connect Four adalah permainan papan klasik untuk dua pemain, di mana pemain secara bergantian menjatuhkan bidak ke dalam kolom papan berukuran 7 kolom x 6 baris. Tujuan utama adalah menjadi pemain pertama yang membentuk empat bidak berurutan secara horizontal, vertikal, atau diagonal.

2. **Tujuan Proyek**

   Proyek ini bertujuan mengimplementasikan Connect Four dalam bentuk aplikasi multiplayer online berbasis client-server, sehingga dua pemain dapat bermain secara real-time dari komputer yang berbeda melalui jaringan.

3. **Fitur Utama**

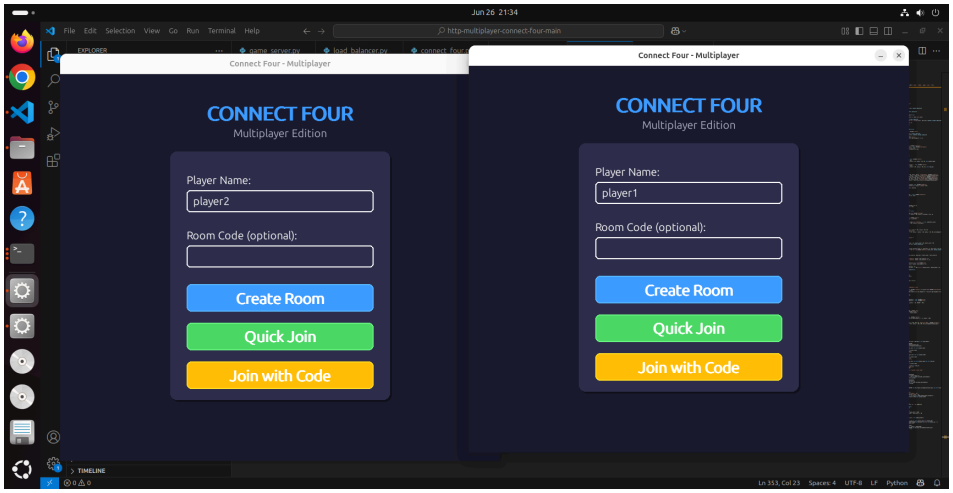
   Game Connect Four ini memiliki beberapa fitur dalam permainannya, yaitu sebagai berikut:

   - **Multiplayer Online:** Dua pemain dapat bermain dari komputer yang berbeda dalam satu jaringan atau internet.
   - **Lobby System:** Pemain dapat membuat room, join room dengan kode, atau quick join ke room yang tersedia.
   - **Load Balancer:** Permintaan client didistribusikan secara otomatis ke dua game server untuk meningkatkan ketersediaan dan skalabilitas.
   - **Sinkronisasi State:** Semua data permainan (room, status pemain, papan, giliran, pemenang) disimpan dan disinkronkan menggunakan Redis.
   - **User Interface:** Client menggunakan Pygame dengan tampilan interaktif, input nama, input kode room, dan notifikasi status.
   - **Notifikasi Real-Time:** Status lobby, giliran, dan hasil permainan ditampilkan secara real-time.

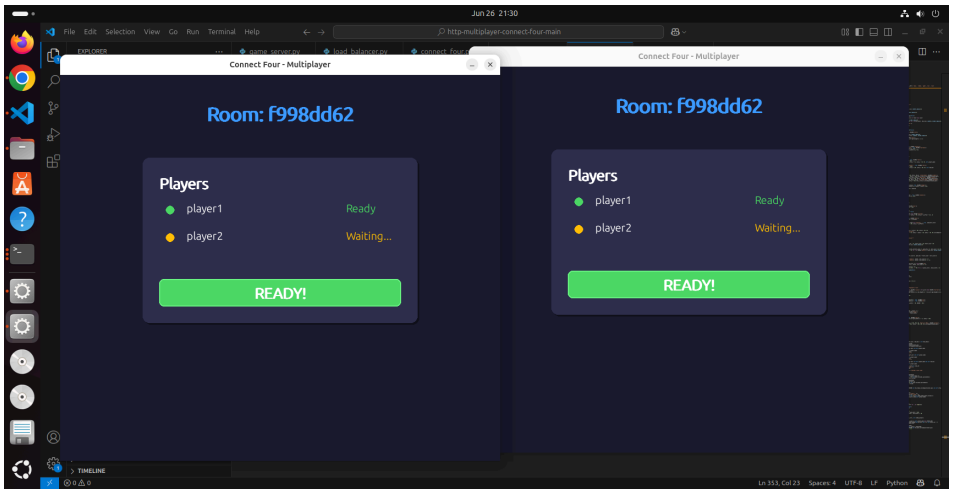
4. **Alur Permainan**

Flow dalam game Connect Four kami memiliki beberapa tahap, berikut adalah flow untuk bermain:

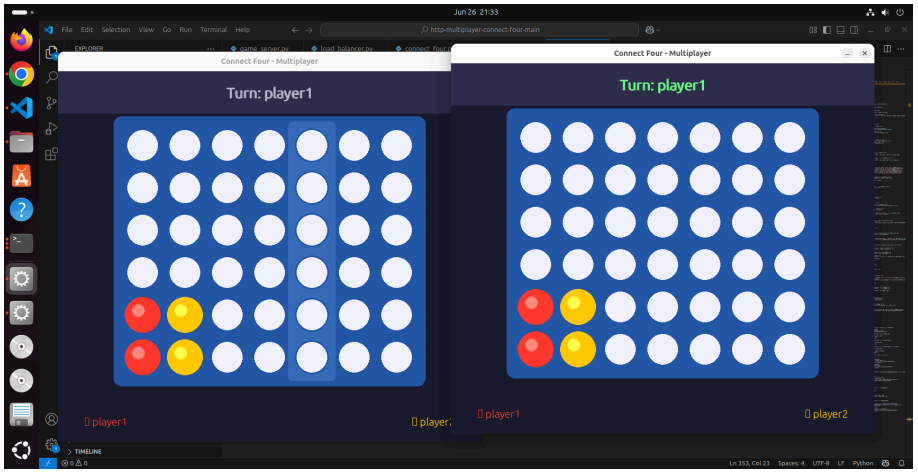
- **Menu Utama:** Pemain memasukkan nama, lalu memilih untuk membuat room, quick join, atau join dengan kode room.



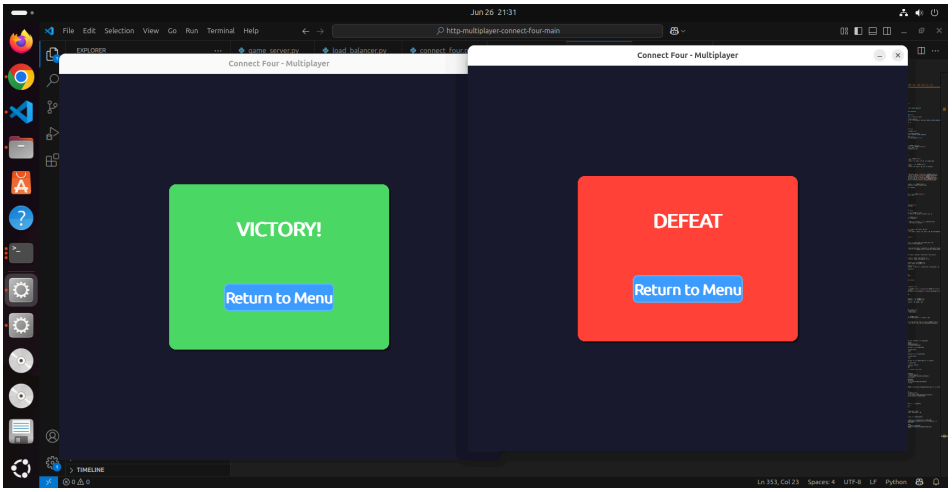
- **Lobby:** Setelah masuk room, pemain menunggu lawan dan menekan tombol "Ready". Permainan dimulai jika kedua pemain sudah siap.



- **Permainan:** Pemain bergantian melakukan langkah dengan mengklik kolom pada papan. Giliran dan nama pemain ditampilkan. Permainan akan berakhir ketika salah satu pemain berhasil membuat 4 baris warna yang seragam.



- **Akhir Permainan:** Jika ada pemenang, akan muncul notifikasi "MENANG!" atau "KALAH!" dan tombol kembali ke menu.



5. **Komunikasi Client-Server**

   Sistem komunikasi pada Game Connect Four ini ada pada Client Side dan Server Side, berikut adalah penjelasannya:

   - **Client:** Mengirim permintaan HTTP ke load balancer di server.
   - **Load Balancer:** Mendistribusikan permintaan ke salah satu game server yang tersedia.
   - **Game Server:** Memproses logika permainan, mengakses/mengupdate data di Redis, dan mengirim respons ke client.
   - **Redis:** Menyimpan seluruh state permainan secara terpusat agar sinkron antar server.

6. **Keunggulan Arsitektur**

   Memiliki beberapa keunggulan dari segi arsitektur yaitu 3 sebagai berikut:

   - **Scalable:** Mudah menambah jumlah game server untuk menangani lebih banyak pemain.
   - **Fault Tolerant:** Jika satu game server gagal, load balancer dapat mengalihkan ke server lain.
   - **Real-Time Sync:** Redis memastikan semua server memiliki data permainan yang konsisten.

7. **Teknologi yang Digunakan**

   Kami menggunakan bahasa pemrograman Python dan beberapa library dari python untuk membantu pengembangan game ini. Selain itu kami juga menggunakan Socketserver, Redis, Docker dan Supervisor.

   - **Pygame:** Untuk tampilan dan interaksi client.
   - **Socketserver:** Untuk membuat server HTTP berbasis TCP dengan dukungan multithread.
   - **Redis:** Untuk penyimpanan state permainan.
   - **Docker:** Untuk containerisasi server, load balancer, dan Redis.
   - **Supervisor:** Untuk mengelola proses multipel di dalam container.

8. **Skenario Penggunaan**

Dalam sebuah sistem permainan jaringan, dua komputer yang berada dalam satu jaringan lokal dapat bermain bersama dengan mudah, cukup dengan mengarahkan client ke alamat IP dari server. Apabila permainan ingin dimainkan melalui internet oleh client di luar jaringan lokal, maka dibutuhkan konfigurasi port forwarding agar koneksi dapat diteruskan ke server yang berada di dalam jaringan privat.

Sistem ini juga mendukung koneksi multi-client, di mana lebih dari dua client dapat bergabung ke server. Namun, untuk menjaga kestabilan dan aturan permainan, hanya dua pemain yang diperbolehkan bermain secara bersamaan dalam satu room. Firewall di tiap mesin harus dimatikan saat memainkan game, karena terdapat resiko bahwa koneksi akan terblokir oleh sistem tersebut.

Informasi Final Project

1. **Github Repository**
   https://github.com/erikplol/http-multiplayer-connect-four
2. **Youtube**
   https://youtu.be/FrPH6QbwHPs

3. **Pembagian Tugas**

Medericus Mundi Miseridityo	Pengembangan Load Balancer
Muhammad Baihaqi Dawanis	Pengembangan Game Server & Redis
Sadam Ali Rafsanjani	Setup Otomasi dengan Docker
Daffa Diandra Rizky A.	Pengembangan Client UI/UX
Mochammad Irham Maulana	Pengembangan Client Networking & Integrasi API

Protokol Multiplayer Game

1. Protokol

Protokol multiplayer pada project ini adalah aturan komunikasi antara client (Pygame) dan server menggunakan protokol HTTP dengan format data JSON. Protokol ini mendefinisikan bagaimana client mengirim permintaan (request) ke server dan bagaimana server merespons (response) permintaan tersebut untuk mengatur alur permainan multiplayer secara sinkron.

Karakteristik Protokol dalam proyek ini adalah sebagai berikut :

- Transport Layer
  - Protocol: HTTP (Hypertext Transfer Protocol).
  - Transport: TCP (Transmission Control Protocol).
  - Port: Load balancer di 5000, game server di 5001, 5002, dan port client sesuai program.
- Format Data
  - Request/Response: Menggunakan format HTTP standar.
  - Payload: JSON (JavaScript Object Notation) untuk data request dan response.
  - Contoh Payload: { "player": "Alice", "room_id": "abcd1234", "col": 3 }
- Metode HTTP
  - POST: Untuk aksi yang mengubah state (create_room, join_room, set_ready, make_move, quick_join).
  - GET: Untuk mengambil status (lobby_status, game_state).
- Endpoint

Endpoint	Method	Payload/Query	Fungsi
/create_room	POST	{player}	Membuat room baru
/join_room	POST	{player, room_id}	Join ke room tertentu
/quick_join	POST	{player}	Join ke room yang tersedia
/set_ready	POST	{player, room_id}	Set status ready di lobby
/make_move	POST	{player, room_id, col}	Melakukan langkah
/lobby_status	GET	room_id (query)	Status lobby (player, ready)
/game_state	GET	room_id (query)	Status papan & giliran

2. Mekanisme Kerja Client-Server

a. Client Mengirim Request

- Client (main.py) mengirim HTTP request ke load balancer (localhost:5000).
- Contoh: POST /make_move, GET /game_state, dll.
-

```python
import requests

SERVER_URL = 'http://localhost:5000'

You, 13 hours ago | 1 author (You)
class HTTPClient:
    def __init__(self, player_name):
        self.player = player_name
        self.room_id = None
        self.session = requests.Session()

    def create_room(self):
        resp = self.session.post(f'{SERVER_URL}/create_room', json=
        {'player': self.player}, timeout=3)
        data = resp.json()
        self.room_id = data.get('room_id')
        return data
```

b. Load Balancer Menerima & Mendistribusikan

- Load balancer (load_balancer.py) menerima request di port 5000.
- Load balancer memilih salah satu game server (misal: 5001 atau 5002) dengan round robin (fungsi get_next_server).
- Load balancer melakukan health check sederhana (ping endpoint /game_state).
- Request diteruskan ke game server terpilih, response dari server dikembalikan ke client (fungsi do_POST).
-

```python
def get_next_server(self):
    global server_index
    with server_lock:
        for _ in range(len(GAME_SERVERS)):
            port = GAME_SERVERS[server_index]
            server_index = (server_index + 1) % len(GAME_SERVERS)
            try:
                requests.get(f'http://localhost:{port}/game_state', timeout=0.5)
                return port
            except Exception: …
        return None

def do_POST(self):
    port = self.get_next_server()
    if port is None: …
    url = f'http://localhost:{port}{self.path}'
    length = int(self.headers.get('Content-Length', 0))
    body = self.rfile.read(length)
    try:
        resp = requests.post(url, data=body, headers={'Content-Type': 'application/json'}, timeout=3)
        self._set_headers(resp.status_code)
        self.wfile.write(resp.content)
    except Exception: …
```

c. Game Server (Thread Pool) Memproses

- Game server (server_thread_pool_http.py) menerima request dari load balancer dan setiap koneksi diproses oleh thread pool.

-
```python
def Server(port):
    the_clients = []
    my_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    my_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    my_socket.bind(('localhost', port))
    my_socket.listen(5)
    logging.info(f"Game server listening on port {port}")
    with ThreadPoolExecutor(10) as executor:
        while True:
            connection, client_address = my_socket.accept()
            logging.info(f"New client: {client_address}")
            p = executor.submit(ProcessTheClient, connection, client_address)
            the_clients.append(p)
            jumlah = ['x' for i in the_clients if not i.done()]
            logging.info(f"Active threads: {len(jumlah)}")
```
- Thread membaca request HTTP.
- Thread memanggil logic game (game_server.py) untuk memproses request (misal: create_room, make_move, dll).
-
```python
def ProcessTheClient(connection, address):
    logging.info(f"Accepted connection from {address}")
    try:
        # Step 1: Read headers until \r\n\r\n
        rcv = b''
        while b'\r\n\r\n' not in rcv:…
        if b'\r\n\r\n' not in rcv:…
        header_end = rcv.index(b'\r\n\r\n') + 4
        headers_raw = rcv[:header_end].decode(errors='replace')
        body = rcv[header_end:]

        # Step 2: Parse Content-Length
        content_length = 0
        for line in headers_raw.split('\r\n'):…
        # Step 3: Read the rest of the body if needed
        while len(body) < content_length:…
        # Step 4: Reconstruct the full HTTP request
        full_request = rcv[:header_end] + body
        request_str = full_request.decode(errors='replace')
        logging.info(f"Request from {address}: {request_str.strip()}")
        hasil = httpserver.proses(request_str)
        hasil = hasil + b"\r\n\r\n"
        connection.sendall(hasil)
        logging.info(f"Response sent to {address} ({len(hasil)} bytes)")
```
- Game server membaca/menulis state room/game ke Redis.
-
```python
def get_room(self, room_id):
    if not room_id:
        return None
    data = redis_client.get(f'room:{room_id}')
    if data:
        return json.loads(data) # type: ignore
    return None
```

d. Redis sebagai Shared State
- Semua game server membaca/menulis data room/game ke Redis.
- Redis berlaku sebagai database, memastikan state konsisten meski request client bisa masuk ke server mana saja.
- Contoh: status board, giliran, status ready, dll.

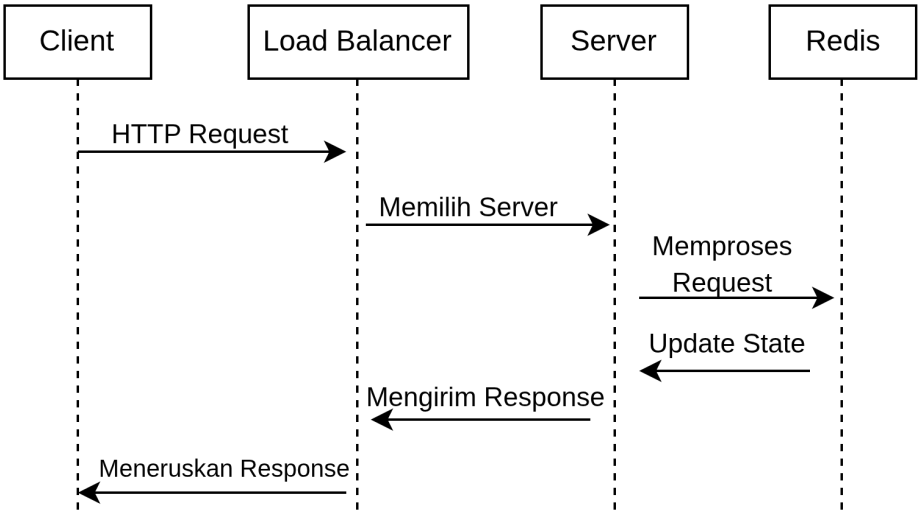
- ```
  def save_room(self, room_id, room):
      redis_client.set(f'room:{room_id}', json.dumps(room))
  ```

e. Response Kembali ke Client

- Setelah game server memproses request dan update state di Redis, response dikirim kembali ke load balancer.
- Load balancer akan meneruskan response ke client.
- Client menerima response (JSON), dan melakukan update UI sesuai data terbaru.
- ```
  # --- Join with Code ---
  elif btn_join.collidepoint(event.pos) and self.player_name and self.room_id:
      self.menu_notification = ""
      self.http = HTTPClient(self.player_name)
      try:
          data = self.http.join_room(self.room_id)
          self.room_id = data['room_id']
          self.state = STATE_LOBBY
      except:
          self.menu_notification = "Failed to join room"
  ```

3. Diagram Alur Komunikasi

Berikut adalah gambar diagram alur komunikasi keseluruhan game Conenct Four mulai dari Client, Load Balancer, Server, hingga Redis.



Arsitektur Implementasi

1. **IP Address Client dan Server**
   A. IP Address Client

      Setiap client (main.py) akan menggunakan alamat IP lokalnya (misalnya, 192.168.0.245) untuk terhubung ke Load Balancer. Karena ini multiplayer, setiap pemain harus menjalankan main.py dari laptopnya masing-masing.

   B. IP Address Server (Load Balancer & Game Servers)

      IP Addres Server mengikuti dimana server tersebut dijalankan. Load balancer dan game server akan dijalankan pada mesin yang sama dengan melalui *containerization* melalui Docker.

      Jika semua bermain di jaringan lokal yang sama (misalnya, semua terhubung ke Wi-Fi rumah yang sama), SERVER_URL akan menjadi http://<ipaddress-server-ygterhubung>:5000.

      Contoh: http://192.168.1.100:5000.

2. **Port TCP/IP yang Digunakan**

   Arsitektur game ini menggunakan model Client-Server terdistribusi dengan bantuan Load Balancer dan Redis untuk penyimpanan data.

   A. Client (main.py, http_client.py):
      - Setiap pemain menjalankan aplikasi klien di perangkatnya masing-masing.
      - Berkomunikasi dengan server melalui HTTP (GET/POST) ke Load Balancer.
      - Setiap klien menggunakan port unik yang ditentukan oleh sistem operasi dan program.

   B. Load Balancer (load_balancer.py):
      - Berfungsi sebagai satu titik masuk (localhost:5000 default) untuk semua permintaan klien.x`
      - Mendistribusikan permintaan ke salah satu Game Server yang tersedia (port 5001, 5002, dst.) menggunakan Round Robin dengan health check.

   C. Game Servers (server_thread_pool_http.py):
      - Beberapa instance dapat berjalan (misal, di port 5001, 5002).
      - Mengelola logika permainan (membuat room, bergabung, gerakan, status game) dan menyimpan status game di Redis.

   D. Redis (Database):
      - Digunakan oleh Game Server untuk menyimpan data game (status room, papan, pemain).
      - Berjalan di port default 6379.
      - Untuk konfigurasi docker akan menjalankan di port 6380 tetapi akan di forward ke port 6379 agar menghindari konflik dengan server redis lokal (jika ada).

3. **Instruksi Menjalankan Server dan Client**
   A. Menjalankan Server
      - **Bangun Docker Image:** docker build -t connect4-server
      - **Jalankan Docker Container :** docker run -p 5000:5000 -p 5001:5001 -p 5002:5002 -p 6380:6379 connect4-server

B. Menjalankan Client

- **Buat Python Virtual Environment (opsional) :** python3 -m venv .venv source .venv/bin/activate
- **Install Requirements :** pip install -r requirements.txt
- **Jalankan Client :** python client/main.py
- Lakukan hal serupa pada mesin/perangkat kedua untuk menjalankan game multiplayernya.